

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Bacharelado em Engenharia de Software

Programação Modular

Trabalho Prático — Sistema de Hospedagem StayHub

Relatório Final

Relatório técnico final apresentado à disciplina de Programação Modular do curso de Engenharia de Software da Pontifícia Universidade Católica de Minas Gerais, consolidando o desenvolvimento do sistema StayHub ao longo das Sprints 1 a 4 do trabalho prático.

Professor: Glender Brás

Belo Horizonte
2026

INTEGRANTES DO GRUPO

- Cauã Thomarco Thomaz Teixeira
- Guilherme Augusto da Silva Machado
- Sofia Figueiredo de Oliveira

SUMÁRIO

- 1 Introdução
- 2 Tecnologias e arquitetura
- 3 Sprint 1 — Modelagem orientada a objetos e interface
- 4 Sprint 2 — API REST e persistência
- 5 Sprint 3 — Exceções, testes e novas funcionalidades
- 6 Sprint 4 — Padrões de projeto
- 7 Testes automatizados
- 8 Demonstração de funcionamento
- 9 Considerações finais
- Referências

1 INTRODUÇÃO

Este documento é o relatório final do trabalho prático desenvolvido na disciplina de Programação Modular, apresentando de forma consolidada o sistema StayHub — uma plataforma de hospedagem inspirada em Airbnb e Booking — construída ao longo de quatro sprints iterativas.

O projeto teve como objetivo aplicar de forma progressiva os principais conceitos do desenvolvimento orientado a objetos moderno: modelagem de domínio, arquitetura em camadas, persistência relacional, tratamento robusto de erros, testes automatizados e, na etapa final, a introdução de padrões de projeto do catálogo GoF. Cada Sprint acumulou requisitos e responsabilidades sobre a base construída na anterior, resultando em um sistema completo, testável e extensível.

O relatório está estruturado da seguinte maneira: a Seção 2 descreve as tecnologias e a arquitetura geral; as Seções 3 a 6 detalham o escopo, as entregas e as decisões técnicas de cada Sprint; a Seção 7 apresenta os testes automatizados; a Seção 8 exemplifica a execução do sistema; e a Seção 9 traz as considerações finais.

2 TECNOLOGIAS E ARQUITETURA

2.1 Stack tecnológica

O StayHub é uma aplicação Java com backend Spring Boot e frontend estático em HTML/CSS. As principais tecnologias empregadas são:

- Java 17 como linguagem-base e Maven como gerenciador de build.
- Spring Boot 3.3 com Spring Web e Spring Data JPA para o backend REST.
- MySQL 8 (perfil padrão) e H2 (perfil de teste em memória).
- JUnit 5 + Mockito para testes unitários e de serviço.
- HTML5, CSS3 e JavaScript nativo para a interface web (stayhub.html).
- PlantUML para modelagem visual do diagrama de classes.

2.2 Arquitetura em camadas

O backend adota uma arquitetura em camadas com separação clara de responsabilidades por pacote, seguindo o padrão MVC estendido comum em aplicações Spring:

Figura 1 — Camadas e pacotes do backend StayHub.

```
controller -> service -> repository -> model
|
+--> exception (excecoes personalizadas + handler global)
+--> notificacao (Observer/Strategy) [Sprint 4]
+--> tarifa      (Strategy/Decorator) [Sprint 4]
+--> log         (Singleton)          [Sprint 4]
```

A camada de controllers expõe endpoints REST anotados com `@RestController`; a camada de services concentra as regras de negócio; os repositórios estendem `JpaRepository` para acesso ao banco; e o pacote model contém as entidades JPA e enumerações. As Sprints 3 e 4 adicionaram novos pacotes especializados (exception, notificacao, tarifa e log) para agrupar responsabilidades transversais.

3 SPRINT 1 — MODELAGEM ORIENTADA A OBJETOS E INTERFACE

A Sprint 1 estabeleceu os alicerces do sistema: a modelagem do domínio de hospedagem e a construção da primeira versão da interface web. Foram identificadas as entidades principais e definidas suas responsabilidades por meio de cartões CRC (Class-Responsibility-Collaborator), disponíveis em Docs/Cartoes CRC.docx.

3.1 Entidades do domínio

- Residencia: unidade física que agrupa quartos disponíveis para locação.
- Quarto: classe abstrata com três especializações — `QuartoIndividual`, `QuartoDuplo` e `QuartoFamilia` — que exemplificam polimorfismo no cálculo do valor da diária.
- Cliente: usuário que realiza reservas, com CPF único e dados de contato.
- Aluguel: transação que associa Cliente e Quarto por um intervalo de datas, com número de hóspedes e opção de berço.

3.2 Polimorfismo no cálculo da diária

Cada subclasse de Quarto define seu próprio calculo de diária, encapsulando as regras específicas do tipo de acomodação. Essa decisão elimina o uso de switch/if aninhados e localiza o conhecimento sobre cada tipo em sua própria classe (Princípio da Responsabilidade Única).

Tabela 1 — Regras de cálculo por tipo de quarto.

Tipo	Fórmula
Individual	$\text{diaria} = \text{valorBase} + (\text{camasSolteiro} - 1) \times 25,00 + \text{adicionais}$
Duplo	$\text{diaria} = \text{valorBase} + \text{adicionalCamaCasal} + (\text{berco} ? 35 : 0) + \text{adicionais}$
Família	$\text{diaria} = \text{valorBase} \times (1 + 0,08 \times \text{numHospedes}) \times (1 - \text{descontoGrupo}) + \text{adicionais}$

Os adicionais comuns (ar-condicionado +R\$ 30, hidromassagem +R\$ 50) e o adicional de conforto do quarto duplo (CASAL = 40, QUEEN = 80, KING = 120) foram encapsulados em métodos utilitários da classe base para evitar duplicação nas subclasses.

3.3 Interface web inicial

A primeira versão do frontend (stayhub.html) apresenta a landing page do sistema, com seções de destaque, catálogo de acomodações e formulários iniciais de reserva. O layout foi

construído com HTML5 e CSS3 puro, sem frameworks, priorizando responsividade e acessibilidade.

4 SPRINT 2 — API REST E PERSISTÊNCIA

A Sprint 2 converteu o protótipo orientado a objetos em uma aplicação Spring Boot completa, com API REST e persistência em banco de dados relacional. Foi introduzida a arquitetura em camadas descrita na Seção 2.2, com separação entre controllers, services, repositories e DTOs.

4.1 Persistência JPA e herança

As entidades foram mapeadas com anotações JPA (`@Entity`, `@Table`, `@Column`). A hierarquia de Quarto emprega a estratégia `SINGLE_TABLE`, com a coluna discriminadora tipo diferenciando Individual, Duplo e Família. Essa escolha simplifica consultas e evita joins em tempo de execução, à custa de algumas colunas nulas por linha — trade-off aceitável para o volume esperado.

4.2 Endpoints REST base

Tabela 2 — Endpoints REST introduzidos na Sprint 2.

Recurso	Endpoint base	Verbos suportados
Residências	/residencias	GET, POST, PUT, DELETE
Quartos	/quartos	GET, POST, DELETE
Clientes	/clientes	GET, POST, PUT, DELETE
Aluguéis	/alugueis	GET, POST, DELETE

4.3 Perfis de execução

Foram configurados três perfis Spring: (i) padrão, com MySQL 8 em localhost:3306; (ii) dev, que recria o banco `stayhub_dev` do zero e popula dados de exemplo; e (iii) h2, com banco em memória e console web para inspeção — usado principalmente em ambiente de teste rápido e demonstrações sem infraestrutura externa.

5 SPRINT 3 — EXCEÇÕES, TESTES E NOVAS FUNCIONALIDADES

A Sprint 3 focou em robustez e verificação. Três frentes foram atacadas em paralelo: exceções personalizadas com tratamento global, cobertura de testes unitários e três novas funcionalidades de negócio (filtro por tipo, cancelamento e histórico).

5.1 Exceções personalizadas

Foi criado o pacote `com.puc.stayhub.exception` com quatro exceções específicas do domínio e um `GlobalExceptionHandler` anotado com `@RestControllerAdvice`, que padroniza

o JSON de erro devolvido pela API:

Figura 2 — Estrutura padronizada do JSON de erro.

```
{
  "timestamp": "2026-06-14T19:59:14.407582700",
  "status": 404,
  "error": "Not Found",
  "message": "Cliente nao encontrado: 999"
}
```

Tabela 3 — Mapeamento das exceções para códigos HTTP.

Exceção	HTTP	Situação em que ocorre
QuartoIndisponivelException	409	Sobreposição de datas com aluguel ativo.
CapacidadeExcedidaException	400	numHospedes excede a capacidade do quarto.
DataInvalidaException	400	Datas nulas, invertidas ou no passado.
RecursoNaoPermitidoException	422	Berço solicitado em quarto individual, entre outros.
MethodArgumentNotValidException	400	Falha de validação Bean Validation (@Valid).

5.2 Testes unitários com JUnit 5 e Mockito

A Sprint 3 entregou 31 testes automatizados cobrindo os quatro pontos exigidos pelo enunciado: cálculo de diária por tipo de quarto, regras de berço, limites de hóspedes e disponibilidade (sobreposição de datas). A suíte de AluguelService utiliza Mockito para isolar as dependências de repositório.

- QuartoIndividualTest — 6 testes de cálculo e regras de berço.
- QuartoDuploTest — 5 testes de adicional de cama e berço opcional.
- QuartoFamiliaTest — 7 testes de desconto de grupo e capacidade.
- AluguelTest — 5 testes de recálculo de valores e cancelamento.
- AluguelServiceTest — 8 testes de criação, cancelamento e histórico com mocks.

5.3 Novas funcionalidades de negócio

Tabela 4 — Endpoints introduzidos na Sprint 3.

Verbo	Endpoint	Descrição
GET	/quartos?tipo=DUPLO	Filtra quartos por tipo (INDIVIDUAL, DUPLO, FAMILIA).
GET	/quartos?residencialId=1&tipo=FAMILIA	Filtra por residência e tipo simultaneamente.
PATCH	/alugueis/{id}/cancelar	Cancela um aluguel (muda status para CANCELADO).
GET	/alugueis/cliente/{id}/historico	Retorna todos os aluguéis do cliente, ordenados por data.

6 SPRINT 4 — PADRÕES DE PROJETO

A Sprint 4 promoveu a evolução arquitetural do sistema por meio da aplicação de padrões

de projeto do catálogo GoF. As subseções a seguir seguem exatamente os cinco tópicos exigidos pelo enunciado: (i) funcionalidades escolhidas; (ii) solução proposta com os padrões e como foram utilizados; (iii) justificativa das escolhas; (iv) utilização do Singleton e justificativa da escolha; e (v) benefícios obtidos com a nova arquitetura.

6.1 Funcionalidades escolhidas

Das seis opções apresentadas no enunciado, o grupo escolheu duas por atacarem problemas reais de acoplamento e falta de extensibilidade que já começavam a aparecer no AluguelService das sprints anteriores:

- Opção 3 — Central de Notificações: o sistema passa a notificar clientes e proprietários quando eventos relevantes ocorrem no ciclo de vida de uma reserva (criação, cancelamento, check-in, check-out, pagamento confirmado). As notificações podem ser enviadas por múltiplos canais (e-mail, SMS, WhatsApp, notificação interna) e novos canais podem ser adicionados sem modificar o código existente.
- Opção 1 — Sistema de Tarifação Flexível: o cálculo do valor da diária passa a considerar regras contextuais como alta e baixa temporada, feriados, promoções temporárias e descontos progressivos para clientes frequentes. Novas regras podem ser criadas e combinadas livremente sem alteração significativa do código existente.

6.2 Solução proposta: padrões e como foram utilizados

Foram aplicados quatro padrões de projeto clássicos do catálogo GoF: Observer, Strategy, Decorator e Singleton. A Tabela 5 sintetiza a aplicação de cada um.

Tabela 5 — Padrões de projeto utilizados na Sprint 4.

Padrão	Funcionalidade	Onde/como é utilizado
Observer	Central de Notificações	CentralNotificacoes publica eventos; NotificadorCliente e NotificadorProprietario são observadores registrados dinamicamente.
Strategy	Central de Notificações	Cada canal (CanalEmail, CanalSMS, CanalWhatsApp, CanalInterno) implementa CanalNotificacao.
Strategy	Tarifação Flexível	RegraTarifa é o contrato; TarifaBase é a estratégia padrão.
Decorator	Tarifação Flexível	RegraTarifaDecorator envolve outra RegraTarifa; permite empilhar AltaTemporada, Feriado, PromocaoTemporaria, etc.
Singleton	Recurso global	CentralNotificacoes, GerenciadorTarifas e LogService (uma única instância por classe).

Na Central de Notificações a combinação Observer + Strategy elimina o acoplamento entre a lógica de negócio (AluguelService) e o mecanismo de comunicação externo. O serviço apenas publica um EventoReserva na CentralNotificacoes; os observadores registrados (cliente, proprietário) recebem o evento e delegam o envio a um

CanalNotificacao injetado em seus construtores. Adicionar um canal Push, Telegram ou Slack no futuro exige apenas criar uma nova classe que implemente a interface, sem tocar em código existente.

Na Tarifação Flexível a combinação Strategy + Decorator resolve a necessidade de aplicar múltiplas regras simultaneamente e em ordem configurável, sem cair em explosão combinatória de subclasses. RegraTarifa define o contrato Strategy (calcular(ctx)); TarifaBase é a estratégia-folha; RegraTarifaDecorator é a classe abstrata que envolve outra RegraTarifa e aplica sua própria transformação sobre o resultado. Cada regra sazonal ou promocional é um decorator concreto (AltaTemporada, BaixaTemporada, Feriado, PromocaoTemporaria, DescontoClienteFrequente). O GerenciadorTarifas monta dinamicamente a cadeia por meio de Function<RegraTarifa, RegraTarifa>.

6.3 Justificativa das escolhas

Observer + Strategy para notificações: o comportamento desejado tem duas dimensões independentes de variação — quem deve ser avisado (cliente, proprietário, futuros interessados como equipe de limpeza) e por qual canal a mensagem trafega. Observer isola a primeira dimensão; Strategy isola a segunda. Ambos são reconhecidos por Gamma et al. (2000) como soluções canônicas para esses cenários, e sua combinação permite que o sistema evolua em cada eixo sem interferência no outro.

Strategy + Decorator para tarifação: Strategy resolve o problema básico de escolher uma regra de cálculo. Contudo, o requisito impõe que múltiplas regras se apliquem simultaneamente e em ordem configurável. Decorator resolve exatamente esse tipo de composição transparente sem herança rígida, mantendo os efeitos combinados sob a mesma interface Strategy. Essa combinação é apontada por Gamma et al. (2000) como padrão recorrente em cálculos financeiros e faturamento.

6.4 Utilização do Singleton e justificativa da escolha

O padrão Singleton foi aplicado em três classes que representam recursos genuinamente globais do sistema. Em nenhum dos três casos o Singleton é o único padrão da solução: ele coexiste com Observer, Strategy e Decorator, cada um resolvendo um aspecto distinto — atendendo à restrição explícita do enunciado.

- CentralNotificacoes: barramento único de eventos. Se houvesse mais de uma instância, os observadores registrados em uma não seriam notificados por eventos publicados na outra, quebrando a semântica do Observer.
- GerenciadorTarifas: mantém o catálogo global de feriados, promoções ativas e a cadeia de decoradores. Duas instâncias produziram cálculos divergentes para reservas

idênticas, comprometendo a coerência do sistema.

- **LogService:** concentra o histórico de registros da aplicação. Um sistema de log fragmentado inviabiliza auditoria e depuração.

A Listagem 1 mostra a implementação de referência, adotada nas três classes, com double-checked locking para garantir criação preguiçosa e segura em ambientes concorrentes.

Listagem 1 — Implementação Singleton com double-checked locking.

```
public class CentralNotificacoes {
    private static volatile CentralNotificacoes instancia;
    private final List<ObservadorReserva> observadores =
        Collections.synchronizedList(new ArrayList<>());

    private CentralNotificacoes() {}

    public static CentralNotificacoes getInstancia() {
        if (instancia == null) {
            synchronized (CentralNotificacoes.class) {
                if (instancia == null) instancia = new CentralNotificacoes();
            }
        }
        return instancia;
    }

    public void publicar(EventoReserva evento) { /* ... */ }
}
```

6.5 Benefícios obtidos com a nova arquitetura

Os principais ganhos observados após a aplicação dos padrões são:

- **Extensibilidade (OCP):** novos canais de notificação, novas regras de tarifação e novos tipos de observadores podem ser incorporados sem alterar código existente — basta criar classes que implementem as interfaces `CanalNotificacao`, `RegraTarifa` ou `ObservadorReserva`.
- **Baixo acoplamento:** `AluguelService` deixou de depender diretamente de e-mail, SMS ou regras de sazonalidade. Ele apenas publica eventos e delega o cálculo ao gerenciador.
- **Alta coesão:** cada classe tem uma responsabilidade única, aderindo ao SRP — Single Responsibility Principle.
- **Facilidade de teste:** as estratégias e decoradores são funções puras (ou próximas disso), o que permite testes unitários independentes, exemplificado pelos testes `CentralNotificacoesTest` e `GerenciadorTarifasTest` entregues nesta Sprint (13 testes verdes).
- **Configurabilidade em tempo de execução:** promoções e feriados podem ser adicionados via endpoints REST (POST `/tarifas/promocoas`, POST `/tarifas/feriados`) sem redeploy.

- Auditabilidade: LogService e o histórico de eventos da CentralNotificacoes produzem um rastro completo das ações do sistema, expostos em GET /notificacoes/logs e GET /notificacoes/historico.
- Isolamento de mudanças futuras: novas dimensões de variação (equipe de limpeza como observador, regra de fim de semana como novo decorator) impactam apenas classes novas, mantendo o núcleo AluguelService intacto.

6.6 Integração no AluguelService e endpoints REST

O AluguelService.criar() foi estendido para: (i) calcular a diária via GerenciadorTarifas; (ii) registrar uma entrada no LogService; e (iii) publicar um EventoReserva do tipo RESERVA_CRIADA na CentralNotificacoes, o que aciona todos os observadores inscritos. Novos endpoints REST expõem as capacidades introduzidas:

Tabela 6 — Endpoints REST da Sprint 4.

Verbo	Endpoint	Descrição
POST	/tarifas/feriados	Cadastra um feriado no catálogo global.
POST	/tarifas/promocoes	Cadastra uma promoção temporária.
POST	/tarifas/simular	Simula o valor final da diária dadas regras ativas.
GET	/notificacoes/historico	Retorna o histórico de eventos publicados.
GET	/notificacoes/logs	Retorna as entradas registradas no LogService.

7 TESTES AUTOMATIZADOS

A suíte de testes cresceu de forma incremental ao longo das Sprints. Ao final do projeto, o total é de 44 testes automatizados executados via mvn test, todos verdes.

Tabela 7 — Distribuição dos testes por sprint.

Sprint	Suíte	Cobertura
Sprint 3	QuartoIndividualTest (6)	Cálculo de diária e regras de berço no quarto individual.
Sprint 3	QuartoDuploTest (5)	Adicional de cama e opção de berço no quarto duplo.
Sprint 3	QuartoFamiliaTest (7)	Desconto de grupo, capacidade e adicionais.
Sprint 3	AluguelTest (5)	Recálculo de valores, cancelamento e diárias.
Sprint 3	AluguelServiceTest (8)	Criação, cancelamento e histórico com mocks.
Sprint 4	CentralNotificacoesTest (6)	Singleton, inscrição, publicação e resiliência a erros.
Sprint 4	GerenciadorTarifasTest (7)	Singleton, feriado, promoção e cadeia de decoradores.

As novas suítes da Sprint 4 verificam simultaneamente o comportamento de Singleton (a mesma instância é sempre retornada), Observer (os observadores inscritos recebem os

eventos), Strategy (o canal correto é acionado) e Decorator (as regras são empilhadas na ordem esperada), confirmando a coerência da implementação com o descrito neste relatório.

8 DEMONSTRAÇÃO DE FUNCIONAMENTO

A execução da aplicação pode ser feita com qualquer um dos três perfis Spring. O perfil h2 é o mais prático para demonstração local por não exigir instalação de MySQL:

Listagem 2 — Comando para executar em modo H2 (banco em memória).

```
mvn spring-boot:run -Dspring-boot.run.profiles=h2
```

```
# Console H2 em http://localhost:8080/h2-console
```

As requisições abaixo exemplificam o fluxo end-to-end das funcionalidades introduzidas na Sprint 4, exercitando todos os padrões de projeto aplicados:

Listagem 3 — Fluxo completo de validação da Sprint 4.

```
# 1) Cadastrar um feriado
POST /tarifas/feriados
{ "data": "2026-12-25" }

# 2) Criar uma promocao "BlackFriday" com 30% de desconto
POST /tarifas/promocoes
{ "nome":"BlackFriday", "percentual":0.30,
  "inicio":"2026-11-20", "fim":"2026-11-30" }

# 3) Simular tarifa para uma diaria em 24/12 -> 26/12
POST /tarifas/simular
{ "valorBase":200.0,
  "dataInicio":"2026-12-24", "dataFim":"2026-12-26",
  "qtdReservasCliente":1 }

# 4) Criar uma reserva (aciona GerenciadorTarifas + LogService +
CentralNotificacoes)
POST /alugueis
{ "clienteId":1, "quartoId":3,
  "dataInicio":"2026-12-24", "dataFim":"2026-12-26",
  "numHospedes":2, "solicitouBerco":false }

# 5) Consultar historico de eventos publicados e log da aplicacao
GET /notificacoes/historico
GET /notificacoes/logs
```

9 CONSIDERAÇÕES FINAIS

As quatro sprints permitiram experimentar de forma prática todo o ciclo de desenvolvimento de um sistema orientado a objetos moderno: modelagem, persistência, testes, tratamento de erros e evolução arquitetural por meio de padrões. O sistema StayHub final possui uma arquitetura em camadas bem definida, cobertura de testes relevante e extensibilidade garantida pelos padrões de projeto aplicados na última sprint.

Do ponto de vista de aprendizado, o projeto reforçou a importância dos princípios SOLID e a diferença entre "código que funciona" e "código que evolui bem". Padrões como Observer, Strategy e Decorator resolveram problemas concretos de acoplamento que já começavam a aparecer no AluguelService, e o Singleton, apesar de controverso, foi justificado por representar recursos genuinamente globais.

Como evolução futura, o sistema comportaria naturalmente: (i) novos canais de notificação (Push, Slack, Telegram); (ii) novas regras de tarifação (fim de semana, black friday recorrente, cupons); e (iii) integração real com serviços de pagamento e e-mail, sem modificação estrutural do núcleo — resultado direto da arquitetura adotada.

REFERÊNCIAS

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

MARTIN, R. C. Código limpo: habilidades práticas do Agile software. Rio de Janeiro: Alta Books, 2009.

FOWLER, M. Refatoração: aperfeiçoando o projeto de código existente. Porto Alegre: Bookman, 2004.

BECK, K. Test-Driven Development: by example. Boston: Addison-Wesley, 2002.

PIVOTAL. Spring Framework Documentation. 2025. Disponível em: <https://spring.io/projects/spring-framework>. Acesso em: jul. 2026.

ORACLE. Java Persistence API Specification. 2019. Disponível em: <https://jakarta.ee/specifications/persistence/>. Acesso em: jul. 2026.